Encapsulation and Access Modifiers

9 min read · Updated 28 Jul 2026 · Topic 9 of 65

Bookmark · Mark done

Why it matters

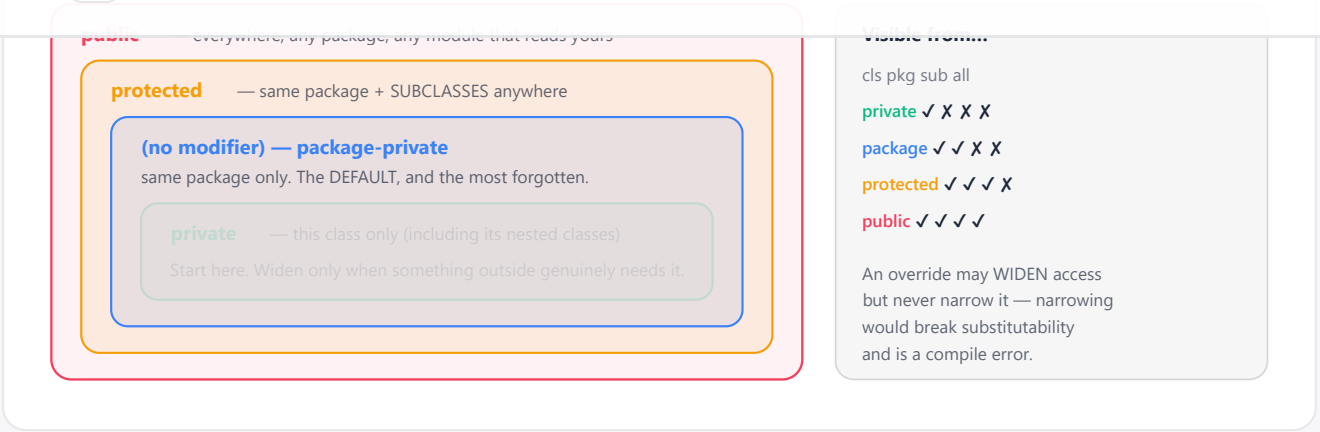
Encapsulation is the OOP pillar most often reduced to a slogan — "make fields private and add getters and setters" — which is not encapsulation at all. A class with a private field, a getter and a setter has exactly the same exposure as a public field, with more typing. Interviewers who care about design ask this question specifically to find out whether you know the difference between hiding a *field* and hiding a *decision*.

The access modifiers themselves also carry two edge cases that come up constantly: what `protected` actually permits, and the fact that the default is not `public`.

The concept, simply

Encapsulation is keeping an object's internal representation private and exposing only operations that maintain its invariants. The point is that the class controls its own state — nobody outside can put it in an invalid condition, and the internals can change without breaking callers.

Java's four access levels are the mechanism:



Widening rings. The default when you write no modifier is package-private — the second-narrowest, not public.

The rule of thumb: **start at `private` and widen only when something outside genuinely needs access**. Every widening is a promise you may have to keep for years.

How it works

Modifier	Same class	Same package	Subclass (other package)	Anywhere
`private`	✓	✗	✗	✗
(none) — package-private	✓	✓	✗	✗
`protected`	✓	✓	✓ *	✗
`public`	✓	✓	✓	✓

The asterisk hides the detail interviewers probe. **`protected` access from another package works only through a reference of the subclass's own type** — a subclass may touch *its own* inherited member, not the same member on an arbitrary instance of the parent. This restriction stops a subclass being used as a back door into unrelated instances.

```java
// package com.app.base
public class Base { protected int value; }

// package com.app.other
public class Sub extends Base {
    void demo(Base otherBase, Sub otherSub) {
        this.value = 1;          // ✓ my own inherited member
        otherSub.value = 2;      // ✓ through a Sub reference
        // otherBase.value = 3; // ✗ compile error – Base reference, other
    }
}
```

Note also that `protected` **is wider than package-private**, not narrower — it adds subclasses *on top of* package access. And `private` is per-*class*, not per-object: one instance can freely read another instance's private fields, which is exactly what makes `equals` and `compareTo` writable.

Code examples

The distinction between real encapsulation and getter/setter theatre:

JAVA Copy

```java
// ✗ Encapsulation in name only. The setter lets anyone reach an invalid s
//    exactly as a public field would – and the getter hands out the live
public class BadAccount {
    private double balance;
    private List<String> transactions = new ArrayList<>();

    public double getBalance() { return balance; }
    public void setBalance(double b) { this.balance = b; }        // balan
    public List<String> getTransactions() { return transactions; } // call
}

// ✓ Real encapsulation: operations, not accessors. The invariant
//    "balance is never negative" is enforced in one place and cannot be d
public class Account {
    private double balance;
```

```
    private final List<String> transactions = new ArrayList<>();

    public void deposit(double amount) {
        if (amount ≤ 0) throw new IllegalArgumentException("deposit must
        balance += amount;
        transactions.add("DEP " + amount);
    }

    public void withdraw(double amount) {
        if (amount ≤ 0) throw new IllegalArgumentException("withdrawal mu
        if (amount > balance) throw new InsufficientFundsException(balance
        balance -= amount;
        transactions.add("WDR " + amount);
    }

    public double getBalance() { return balance; }

    public List<String> getTransactions() {
        return List.copyOf(transactions);       // defensive copy — caller
    }
}
```

The leaky-getter bug is worth showing on its own, because it is so common in real code:

JAVA Copy

```
Account acc = new Account();
acc.deposit(100);

// With BadAccount, this compiles and silently destroys the audit trail:
// acc.getTransactions().clear();

// With Account it throws UnsupportedOperationException — the copy is immu
// Same lesson as Chapter 1's pass-by-value: handing out a reference to yo
// internal object hands out control of it.
```

And mutable fields need protecting on the way *in* too:

JAVA Copy

```
    public DateRange(Date start) {
        this.start = new Date(start.getTime());    // copy IN, or the calle
    }                                              // can mutate our state

    public Date getStart() {
        return new Date(start.getTime());          // copy OUT
    }
}
// Modern answer: use java.time.LocalDate/Instant, which are immutable, an
// the whole defensive-copying dance disappears.
```

Under the hood

Access modifiers are enforced twice: by `javac`, and again by the JVM at link time — the verifier rejects a class file that accesses a member it should not see. So they are a genuine boundary, not a compile-time convention. What they are *not* is a security barrier against a determined caller in the same JVM: reflection can historically bypass them with `setAccessible(true)`, which is how frameworks like Jackson and Hibernate populate private fields.

That loophole narrowed considerably. **Java 9's module system** added a second, stronger layer: a package is only visible outside its module if the module `exports` it, and reflective access needs `opens`. Since **Java 16 strong encapsulation is the default** for JDK internals, which is why `sun.misc.Unsafe`-style access and some older libraries broke on newer Java. "Public" now means public *and* exported.

Records (Chapter 10) are worth mentioning here as the modern answer to the data-carrier case: `record Point(int x, int y) { }` gives you final fields, accessors, `equals`, `hashCode` and `toString`, with no setters by construction. When someone asks about getters and setters, saying "for immutable data carriers I would reach for a record" is a good, current answer.

Common mistakes

- **Equating encapsulation with getters and setters.** A public setter for every private field is a public field with ceremony.
- **Returning the live internal collection.** Callers can mutate your state. Return `List.copyOf( ... )` or an unmodifiable view.
- **Storing a mutable argument directly.** Copy on the way in as well as out — or use immutable types and skip both.
- **Assuming no modifier means public.** It means package-private. This surprises people coming from C#.
- **Thinking `protected` is narrower than package-private.** It is wider — package access plus subclasses.
- **Making fields `protected` for subclasses.** That freezes your representation into the subclass contract forever. Expose a `protected` method instead.
- **Believing `private` is per-object.** It is per-class; two instances of the same class see each other's privates.

Interview lens

> **Q: "What is encapsulation?"**
> A: Keeping an object's representation private and exposing only operations that preserve its invariants, so the class is the sole authority on its own state and the internals can change without breaking callers. Private fields are the mechanism, not the goal — a class with a setter for every field is not encapsulated, because any caller can still drive it into an invalid state.

> **Q: "What is the default access modifier?"**
> A: Package-private — visible within the same package only. It is the second-narrowest level, not `public`, and it is genuinely useful: it lets classes that collaborate closely inside a package cooperate without exposing anything to the outside world.

A: Same package, plus subclasses in any package. The subtlety is that cross-package subclass access only works through a reference of the subclass's own type — a subclass can touch its own inherited member, not that member on an arbitrary parent instance. That stops subclassing being used as a back door into unrelated objects.

Q: "Can a class be private?"
A: A top-level class can only be `public` or package-private. Nested classes can be any of the four — a `private static` nested class is a common way to keep an implementation detail genuinely invisible.

Q: "How do frameworks set private fields?"
A: Reflection with `setAccessible(true)` — which is how Hibernate and Jackson populate entities without demanding setters. Access modifiers are checked by the compiler and the JVM verifier, but they were never a security boundary within a JVM. Since Java 9's module system, and strong encapsulation by default from Java 16, that access needs the package to be `opens`-ed, which is why some older libraries broke on newer Java.

Q: "Is returning a getter for a list a problem?"
A: Yes, if it returns the live collection — the caller receives a reference to your internal object and can clear or reorder it, bypassing every invariant you enforce. Return `List.copyOf( ... )` for a snapshot, or an unmodifiable view if the cost of copying matters and the caller understanding it is live is acceptable.

Cheat-card

- Encapsulation = **hide the representation, expose operations that keep invariants**. Not getters/setters.

- Four levels: `private` ⊂ package-private (the default) ⊂ `protected` ⊂ `public`.
- `protected` = package + subclasses; cross-package only via a **subclass-typed reference**.
- `private` is **per class, not per object** — that is why `equals` can read the other instance.
- Return `List.copyOf( ... )`, never the live collection. Copy mutable arguments **in and out**.
- Prefer immutable types (`java.time`, records) and the copying problem disappears.
- Reflection bypasses modifiers; **modules** (9) and strong encapsulation (16) narrowed that.
- Top-level classes: only `public` or package-private.

QUICK CHECK — no marks, just for you.

1. What is the default access level when you write no modifier?

- ◯ protected
- ◯ package-private — visible only within the same package
- ◯ private
- ◯ public

Check answer

2. A subclass in a different package tries to access a protected field on a Base-typed reference to another object. What happens?

- ◯ It compiles only if the field is also final
- ◯ It compiles — protected means any subclass can access it anywhere

It is a compile error: cross-package protected access works only through a reference of the subclass's own type

- ◯ It compiles but throws IllegalAccessError at run time

Check answer

3. A class has a private list and a getter that returns it directly. Why is that a problem?

- ◯ It causes a memory leak
- ◯ It is not a problem — the field is private, so the object is encapsulated
- ◯ It prevents the class from being serialised
- ◯ The caller receives a reference to the live internal collection and can clear or reorder it, bypassing every invariant the class enforces

Check answer

‹ Previous
Abstract Classes vs Interfaces

Next ›
equals() and hashCode(): The Contract

LEARN MORE

ARTICLES & DOCS

- Controlling access to members of a class
  docs.oracle.com
- Determining accessibility (JLS §6.6)
  docs.oracle.com

VIDEOS

"encapsulation access modifiers" — video explainers

YouTube

encapsulation access modifiers interview questions

YouTube

Hand-picked references for this lesson.